ARCHITECTURE REVIEW / PRODUCTION ASSISTANT

Cascade hardening verification

What the **muse_hardening** branch fixed, what remains exposed, and how classic storyboard and node workflows remain two views of one graph.

Branch head reviewed: 8a7fe484

13 October 2026

The foundation is real; the migration is not finished

Verdict: meaningful improvement, not yet “done.”

The branch adds several correct architectural seams, but none of the six original recommendations is fully closed. Five are partly implemented; narrow renderer updates remain the largest missing piece.

5
recommendations partially implemented

1
recommendation still materially open

0
recommendations ready to retire from the risk register

What is genuinely better

Main-process job ledger

There is now a credible authority for generation lifecycle state instead of relying only on renderer-local spinners and queues.

Narrow commands exist

The branch establishes the right command-oriented pattern, even though node-graph field edits have not moved onto it yet.

Pipe resolver and write-through

Storyboard and canvas output selection are closer to a shared model, with explicit resolution and compatibility write-through.

Machine-readable guardrails

An ownership table and a node-kind completeness check now make hidden architectural assumptions visible to the type system.

Plain English: the new load-bearing beams are in place, but several old routes still go around them. The next pass should remove bypasses rather than add another abstraction layer. Classic storyboard and node view must also become two editors over the same stored graph, with automation visible as real nodes and edges.

SIX-POINT SCORECARD

The branch addresses every area, but closes none completely

Original recommendation	Status	What is now present	What still blocks closure
Stable media identity	PARTIAL	Media IDs are recorded.	Consumers still resolve primarily from paths; IDs remain parallel bookkeeping.
One stored graph	PARTIAL	Node-kind completeness is checked.	The canvas can still reconstruct meaning from flat legacy fields, and paired views can drift unless both commit through one graph authority.
One job ledger	PARTIAL	A central ledger and lifecycle structure exist.	`cascade_*` agent generations bypass it, so some jobs remain invisible to the authority.
Targeted updates	OPEN	A narrow-command pattern exists.	Node-graph field edits still travel through whole-project saves, preserving the stale-snapshot race.
One selected output	PARTIAL	Pipe resolution and write-through reduce storyboard/canvas drift.	Portable document format (PDF) export bypasses the pipe and may miss piped-only selections.
Field ownership rule	PARTIAL	An ownership table and exhaustive typing exist.	The normal test command does not run type checking, so a mismatch may not fail continuous integration.

> **Most important remaining correctness risk:** a renderer can still save a stale whole-project snapshot after a generation completes. That can overwrite newer node-graph state even though the ledger and ownership model are otherwise correct.

Review scope: static review of branch head 8a7fe484. This report does not claim runtime profiling or a packaged-app test run.

CORRECTNESS GAPS

Six bypasses keep the old failure modes alive

01 Move node-graph edits off whole-project saves

Node moves, resize, edge changes, parameter edits, and selection should become narrow, typed commands. This is the only item still materially open and the best protection against completed generations disappearing after unrelated edits.

02 Make the stored graph the only post-migration read model

Normalize old projects once. After normalization, the canvas should read stored nodes and edges only. Re-deriving from flat fields keeps two graph definitions alive and makes second-load behavior uncertain.

03 Route every generation through the ledger

Bring `cascade_*` agent-triggered generations into the same create, run, succeed, fail, cancel, and retry lifecycle used by Production Assistant actions.

04 Resolve media by ID in real consumers

A stable ID provides no safety while storyboard, playback, history, and export still use path-bearing fields. Convert one read path at a time, keeping path fields as compatibility data only.

05 Make export use the selected pipe output

PDF export should call the same resolver as the storyboard and canvas. Otherwise a shot that looks correct in the app can export with a blank panel when its current image exists only in the pipe.

06 Run type checking in the standard test path

The ownership and node-completeness guards are valuable only if routine automation executes them. Add type checking to the test or continuous-integration command that gates merges.

PERFORMANCE DIAGNOSIS

The slow path repeats project-scale work for every media request

Caching already exists. The problem is what happens before the cache can help.

Production Assistant already uses thumbnail caches, a parsed-project cache, and lazy image loading. The dominant inefficiency is that each image or video request reloads the project, makes a full copy, and checks the project's media files on disk merely to recover the project folder and locate one asset.

Page requests many thumbnails → **Each request reloads and validates the project** → **Cost multiplies with both card count and total media**

Example shape of the problem

A page with 200 thumbnails can trigger roughly 200 full project reloads. As the project gains media, each reload becomes more expensive, so the slowdown accelerates in exactly the projects that need the interface most.

Why "add another cache" is not first

Caching thumbnails cannot remove repeated project loading that occurs on the request path. The safer fix is to stop doing unrelated work, then tune cache policy after measurement.

> **Low-risk design rule:** a media-serving request should resolve a trusted project folder and one validated relative asset path. It should not hydrate, clone, merge, or repair the complete project document.

PERFORMANCE PLAN

Five contained changes should improve page loads without changing project meaning

1. ## Add a folder-only media lookup

   Provide a narrow, validated route from project identity to its folder, then resolve the requested asset inside that folder. Do not load the full project. Preserve traversal checks and existing missing-file behavior.

2. ## Build the shot lookup once per thumbnail batch

   Create a shot-ID map once, then reuse it for every item in the batch. Avoid scanning the complete shot list separately for each thumbnail.

3. ## Use the existing batch path for animatic thumbnails

   Storyboard thumbnails already demonstrate batching. Route the animatic page through the same service rather than issuing one project-scale lookup per card.

4. ## Skip off-screen layout and paint

   Apply `content-visibility: auto` with a representative `contain-intrinsic-size` to long card lists. This is a browser rendering hint, not a data-model change.

5. ## Bound thumbnail memory by bytes

   Keep the existing item-count cap, add an estimated byte cap, and revoke or release evicted resources. Large thumbnails vary too much for count alone to control memory.

> **Do not lead with:** full list virtualization, project-document splitting, or changes to card memoization and state ownership. Those can help later, but they change interaction behavior and carry a larger regression surface.

Also remove the dead image handler identified during review after confirming no packaged or legacy route still calls it.

VERIFICATION PLAN

Close the architecture gaps with behavior-level tests

Concurrency and saves

- Start a generation, then edit a prompt and move a node before completion.
- The result, prompt, and layout all survive save, close, and reopen.
- A stale destructive command returns a recoverable conflict.

Graph migration

- An old project normalizes to stored nodes and edges.
- The second load is identical and does not re-derive the graph.
- Unknown node data survives load and save.

Job authority

- Agent and user-interface generations appear in one ledger.
- Main and detached windows show the same state after reopening.
- Cancel and retry preserve attempt history and select only the intended result.

Selection and export

- Select an older pipe result; canvas and storyboard update together.
- Reload and export without compatibility fields populated.
- The exported panel is the exact output shown in both views.

Guardrails

- Add a temporary shot field; the normal test command fails until ownership is declared.
- Add an incomplete node kind; the same command fails with a useful message.
- Packaged-app smoke testing follows main-process changes.

Performance

- Instrument project loads and media requests before and after.
- A 200-thumbnail page performs one project-folder resolution, not 200 project reloads.
- Cold and warm loads preserve identical thumbnails, errors, and selection state.

Measure before and after on one representative large project

Metric	Why it matters	Pass signal
Project load count	Confirms the hot path is removed	Near constant per page
Time to useful thumbnails	Reflects perceived page speed	Clear reduction
Memory after page switching	Checks cache pressure	Stabilizes under byte cap

TWO-VIEW INTERACTION CONTRACT

Classic storyboard and node view must edit one canonical graph

Product rule

Classic storyboard is an assisted editor over the node graph, not a second implementation. It may choose sensible defaults and automate nodes and edges. Node view exposes the same state for direct control. Either route must produce a project the other can continue editing.

Classic storyboard: intent first

- Translate each user action into the normal typed graph commands.
- Create or update the required backing nodes, inputs, edges, and selected output automatically.
- Keep provenance visible so opening node view explains the result.
- Never require node view to finish an ordinary storyboard task.

Node view: structure first

- Let users create, connect, branch, and tune graph elements explicitly.
- Do not infer an output connection merely because a generation completed.
- Reflect the connected output and compatible settings back in storyboard.
- Never let storyboard opening flatten or rewrite a more complex graph.

Non-negotiable invariants

- **One persisted truth:** nodes, edges, selected output, media identity, and job state come from shared authorities rather than per-view copies.
- **Immediate parity:** save, reopen, undo, retry, export, and detached windows resolve the same result from either entry point.
- **Visible automation:** classic actions may automate graph work, but may not hide writes that node view cannot represent.
- **Complexity is preserved:** when a graph cannot be fully expressed by classic controls, storyboard shows the resolved shot and a clear limited-edit state; it does not simplify the graph destructively.

Mental model: classic storyboard is the guided control panel; node view is the expert control panel. The engine and project state underneath are identical.

TWO-VIEW COVERAGE MAP

Apply the same contract to every shared shot operation

User intent	Classic storyboard automation	Node-view behavior and parity check
Generate an image or video	Create or update the correct generation node, inputs, parameters, result history, and visible output edge.	Every generation stays local unless explicitly wired; storyboard follows only the connected output.
Edit an image	Create an Edit Image node from the current source and wire its successful result to Frame Output.	Keep the result on the edit node until the user connects it.
Change prompt, model, or settings	Update the backing node through typed commands and regenerate only when the action requests it.	Compatible node changes appear in classic controls; unsupported complexity is preserved, not flattened.
Add a reference or source	Materialize the source as a graph input or reference and create the required connection.	Inputs and edges remain explicit; classic shows the effective references of the selected path.
Choose an alternate or older result	The shot's left/right arrows select among real image and video generations, then update the visible output connection.	The connected result - image or video - drives storyboard, playback, reopen, and export.
Retry, cancel, or inspect progress	Use the same ledger lifecycle and attempt history.	Both views and windows show the same current attempt and outcome.
Delete, reset, undo, or redo	Issue reversible graph commands that preserve unrelated branches.	The same history operation restores identical nodes, edges, selection, and frame.

> **Do not add view-specific shadow fields.** A classic control may present a simplified value, but its committed change must target the canonical graph. Node view must not maintain a separate output or parameter state.

Round-trip acceptance path

1. Build and edit a shot entirely in classic storyboard.
2. Open node view and verify that every effective input, operation, result, and output connection is visible.
3. Branch and fine-tune in node view without selecting the new branch.
4. Return to storyboard: the frame remains unchanged.
5. Connect the branch to Frame Output: storyboard, playback, reopen, and export all switch together.

> **Coverage rule:** any new shot action that exists in both surfaces must define its classic automation, node-view manual behavior, canonical commands, and round-trip test before release.

Output binding must follow the surface that initiated any generation

Required behavior

The classic shot panel is the guided workflow: a successful image generation, image edit, or video generation becomes the shot's selected Frame Output automatically. The node view is explicit: every generated result stays on its own node unless the user wires it to Frame Output.

Classic shot panel / storyboard view

- Create or update the appropriate backing image, edit, or video node in the stored graph.
- On success, create or select a visible edge from that result to Frame Output.
- Keep image and video generations in shot history; the left/right arrows may select any prior result and update the real output connection.
- Opening node view shows exactly where the selected frame came from.

Node view

- Store and display each generated image or video on the node that produced it.
- Do not create an edge or change Frame Output automatically for any generator type.
- An existing Frame Output remains unchanged while the new node is unwired.
- Only an explicit user connection may route that result to Frame Output.

Implementation constraint

Pass an explicit initiation context into the shared generation-completion command. For every media-generating node, node-view completion commits only the node result, history entry, and ledger state. Classic-panel completion also invokes the normal graph connection and output-selection commands, so convenience behavior and arrow-based history selection are represented by real graph state.

No hidden compatibility shortcut. The classic panel may automate wiring, but it must not bypass graph persistence or mutate a flat Frame Output field behind the graph. Renderer-only conditions would let views, export, and detached windows disagree.

Invariant: generation type does not change the rule. Classic storyboard may auto-connect images, edits, and videos; node view never auto-connects any generated media.

Tests must distinguish classic-panel intent from node-view intent

Regression matrix

Scenario	Expected graph and frame behavior	Persistence check
Classic image generation or edit	Backing node updates; a visible edge selects its result as Frame Output	Same after reopen and export
Classic video generation	Backing video node and explicit Frame Output edge are created or selected	Node view reveals both after reopen
Classic arrows: video back to older image	History selection updates the real output connection; the older image becomes active	Storyboard, playback, reopen, and export agree
Node-view image, edit, or video node; unwired	Node result changes; Frame Output and selected output remain unchanged	Same after reopen
Node-view generated media; explicitly wired	Frame resolves to that image or video through the edge	Same after reopen and export
Node-view wire removed, then regenerate	New result stays local; frame is not overwritten	Same after reopen
Mixed image/video branches	Only the connected branch drives Frame Output	Canvas, storyboard, playback, and export agree

State assertions

- Every result belongs to a backing graph node.
- Classic-panel auto-selection creates or updates a real edge.
- Node-view generation never adds an edge implicitly.
- Failed or canceled jobs leave the prior frame selection intact.

Cross-surface checks

- Canvas and storyboard show the same selected frame.
- PDF export follows the connected output.
- Detached and main windows agree after completion.
- Undo, retry, and reopen preserve the initiating workflow's contract.

Definition of done: classic image, edit, and video actions materialize nodes and visible output wiring, including arrow-based history changes. Every node-view generation remains local until the user explicitly connects it.

LEGACY-PROJECT MIGRATION

Migrate, verify, then remove compatibility reads

Old files need an explicit bridge to the new authority model.

The branch already covers important cases: it can build a stored graph when one is absent, move old board artwork into the pipe model, and back-fill a missing media registry. Keep those transforms, but treat them as the beginning of a versioned migration pipeline rather than permission to remove every legacy field immediately.

Open a copy and detect legacy shape → **Normalize and validate all references** → **Commit once; second load is unchanged**

Legacy state	Required migration	Removal gate
No stored graph	Create stored nodes and edges from the legacy shot fields; preserve unknown node data.	Old-project fixtures render identically after two reopen cycles; the canvas no longer reconstructs the graph.
Artwork without pipe state	Create the equivalent pipe result and selected-output reference without changing the visible choice.	Storyboard, canvas, reload, and PDF export all show the same panel with legacy reads disabled.
Path-only media references	Create registry entries and attach stable IDs while retaining paths as compatibility metadata.	All active consumers resolve by ID; missing or moved media remains recoverable rather than being dropped.
No schema or revision marker	Assign the current schema baseline and save revision only after every transform validates.	Stale-save and failed-migration tests leave the original project usable and unchanged.

Compatibility rule: legacy parsing may remain inside the migrator while this gate is being proved. After migration succeeds, canvas, export, playback, and storyboard should read only the new model.

Protect the original and prove the second load

Migration safety rules

- **Version the file format.** Run ordered migrations from the detected schema version; do not infer and rewrite on every open.
- **Make each step idempotent.** A crash, retry, or second load must not duplicate nodes, media entries, or pipe results.
- **Validate before replacement.** Check graph edges, media IDs, selected outputs, and ownership invariants before atomically replacing the project file; retain a recoverable pre-migration copy.
- **Preserve unresolved data.** Missing media, unknown node kinds, and extra fields should survive round-trip migration with a useful warning.
- **Fail without damage.** If any transform or invariant fails, keep the source project untouched and report the failing migration step.

Recommended rollout

1. ### Build a legacy fixture corpus

   Include representative projects from every supported historical format, plus missing-media, unknown-node, and partially migrated cases.

2. ### Run migrations on copies

   Open, migrate, save, close, and reopen each fixture. Confirm that the second load produces no further document change.

3. ### Compare behavior, not just JSON

   Check shot, node, and edge counts; selected outputs; media references; storyboard appearance; playback; and PDF export.

4. ### Remove fallbacks behind the same gate

   Disable runtime legacy reads and rerun the corpus. Release only when migrated files behave identically through the new authorities.

> **Release gate:** every supported legacy fixture migrates once, reopens unchanged, preserves unresolved data, and passes view, playback, and export checks with runtime compatibility reads disabled.

CODING-AGENT HANDOFF

Finish authority boundaries first; optimize the hot path second

Recommended order

1. **Make the safeguards unavoidable.** Run type checking in the standard test and continuous-integration path so ownership and node-completeness failures block merges.
2. **Remove the worst stale-save surface.** Convert node move, resize, edge, parameter, and output-selection edits to typed commands with revision checks.
3. **Make the two-view contract universal.** Treat classic storyboard as guided automation over the same canonical graph exposed by node view. Cover generation, edits, settings, references, result selection, job controls, deletion, and undo - not only Frame Output.
4. **Make output binding contextual and media-agnostic.** Image, edit, and video results created in node view stay local until explicitly wired. The same actions in classic storyboard create or update a backing node, automate a real edge to Frame Output, and retain prior image/video results for arrow navigation.
5. **Finish and gate legacy migration.** Version the format; normalize graph, pipe, and media identity on a recoverable copy; verify a second load is unchanged; then remove runtime compatibility reads.
6. **Close the remaining bypasses.** Put `cascade_*` generations in the ledger; make PDF export call the pipe resolver; and switch verified consumers from paths to media IDs.
7. **Apply the low-risk performance pass.** Folder-only media lookup, one-pass shot maps, animatic batching, off-screen rendering hints, and a byte-bounded thumbnail cache.

> **Sequence rule:** establish one graph authority and the cross-view contract before optimizing view-specific paths. Performance work should not preserve a second source of truth.

Preserve one graph across both workflows

Copy-ready implementation request

Finish `muse_hardening` without redesigning the app. Replace whole-project node edits with typed commands and put the type guards in the normal test gate. Make classic storyboard a guided editor over the same canonical graph used by node view: every classic generate, edit, setting, reference, selection, retry, delete, and undo action must materialize as normal graph or ledger commands that node view can inspect and continue. Preserve complex node graphs when classic controls cannot express them. For every image-generation, image-edit, and video-generation node, node-view completion updates only that node and never changes Frame Output until the user wires it. In the classic shot panel, create or update the appropriate backing node and automatically add a real, visible Frame Output connection through normal graph commands. Preserve all prior image and video generations in shot history; the shot's left/right arrows must select those real results and update the same output connection, including switching from video back to an older image. Add a versioned, idempotent migration for legacy graph, pipe, and media references; validate it on a recoverable copy and remove compatibility reads only after two-load fixture tests pass. Route `cascade_*` generations through the ledger, make export and all views resolve one selected pipe output, and move verified consumers from paths to media IDs. For speed, use folder-only media lookup, one shot map per batch, animatic batching, off-screen containment, and a byte-bounded cache.

Definition of done

- A classic-to-node-to-classic round trip preserves graph structure, selected output, and visible shot state.
- A complex node graph is never flattened merely by opening or editing in classic storyboard.
- An unrelated renderer edit cannot erase a completed generation.
- An unwired image, edit, or video result created in node view never changes Frame Output.
- A classic-panel image, edit, or video action creates or updates a backing node and a visible output edge.
- The classic shot arrows can move between prior image and video generations without creating hidden state.
- Every generation has one visible ledger lifecycle.
- Every surface and export resolves the same selected output.
- Legacy projects migrate once, reopen unchanged, and retain unresolved data safely.
- The canvas reads one stored graph after migration.
- Normal tests fail on an unowned field or incomplete node kind.
- Media-heavy pages stop reloading the project per thumbnail.

Bottom line: keep the branch. It is the right direction. Treat it as a strong foundation and close the issues above before declaring the architecture hardened.